

UNIVERSIDADE FEDERAL DE SANTA CATARINA

CAMPUS REITOR JOÃO DAVID FERREIRA LIMA

Gabriel Salmoria

Lucas de Souza Vieira

IBE Mail: Criptografia Baseada em Identidade com o Esquema de Boneh-Franklin

Florianópolis – SC

2026

SUMÁRIO

Sumário **1**

1 INTRODUÇÃO **3**

2 FUNDAMENTAÇÃO TEÓRICA **4**
2.1 CRIPTOGRAFIA BASEADA EM IDENTIDADE 4
2.1.1 Definição Formal 4
2.1.2 Noções de Segurança 4
2.1.3 Comparação com PKI e Custódia de Chaves 5
2.2 FUNDAMENTOS MATEMÁTICOS 5
2.2.1 Curvas Elípticas e Grupos 5
2.2.2 Emparelhamentos Bilineares 5
2.2.3 O Problema Bilinear de Diffie-Hellman (BDH) 6
2.3 O ESQUEMA FULLIDENT 6
2.3.1 Funções de Hash 6
2.3.2 Algoritmos 6
2.3.2.0.1 Setup. *6*
2.3.2.0.2 Extract. *7*
2.3.2.0.3 Encrypt. *7*
2.3.2.0.4 Decrypt. *7*
2.3.3 Corretude 7
2.3.4 Identidades Temporais e Revogação 8
2.4 EXEMPLO ILUSTRATIVO 8

3 EXPERIMENTO **10**
3.1 DESCRIÇÃO DO EXPERIMENTO 10
3.1.1 O que demonstramos 10
3.1.2 Arquitetura da Aplicação 10
3.1.3 Formato de Mensagem (`.ibemail`) 10
3.1.4 Registro de Identidades (`registry.dat`) 11
3.2 PSEUDOCÓDIGO 11
3.2.1 Inicialização do Sistema (Setup) 11
3.2.2 Extração de Chave Privada (Extract) 12
3.2.3 Cifragem de Mensagem (Encrypt) 12
3.2.4 Decifração de Mensagem (Decrypt) 12
3.2.5 Fluxo de Uso da Interface 13

4 IMPLEMENTAÇÃO **14**

4.1 BIBLIOTECAS UTILIZADAS 14
4.2 ESTRUTURA DO PROJETO 14

5 RESULTADOS E VALIDAÇÃO PRÁTICA 16
5.1 REGISTRO E AUTORIDADE DE CHAVES (PKG) 16
5.2 TROCA DE MENSAGENS 17
5.3 SEGURANÇA IND-ID-CCA2 19
5.4 IDENTIDADES TEMPORAIS (CIFRAGEM PARA O FUTURO) . . . 21

6 CONCLUSÃO 23

REFERÊNCIAS 24

1 INTRODUÇÃO

A criptografia de chave pública tradicional usa uma infraestrutura de certificados digitais, a PKI (*Public Key Infrastructure*). Para cifrar uma mensagem a alguém, é preciso ter o certificado dessa pessoa, emitido por uma Autoridade Certificadora de confiança (MENEZES, 2009). Na prática isso gera complicações: gerenciar certificados, lidar com revogação e, principalmente, precisar do certificado do destinatário antes de qualquer comunicação.

A Criptografia Baseada em Identidade (IBE, do inglês *Identity-Based Encryption*), proposta por Shamir em 1984 e construída por Boneh e Franklin em 2001 (BONEH; FRANKLIN, 2001), resolve isso de outra forma. A chave pública de um usuário é simplesmente sua identidade, por exemplo seu endereço de e-mail. Qualquer um pode cifrar para `bob@ufsc.br` sem nunca ter interagido com Bob e sem precisar de certificados. Um *Private Key Generator* (PKG) é quem gera e entrega a chave privada ao usuário, após verificar sua identidade.

O esquema proposto por Boneh e Franklin, chamado **FullIdent**, usa emparelhamentos bilineares sobre curvas elípticas supersingulares e é provadamente seguro sob a noção IND-ID-CCA2 no modelo do oráculo aleatório, com base na dificuldade do problema de Diffie-Hellman Bilinear (BDH) (BONEH; FRANKLIN, 2001).

Neste trabalho, implementamos o **IBE Mail**: um cliente de e-mail com interface gráfica que mostra o funcionamento do FullIdent de ponta a ponta. A aplicação roda com múltiplas identidades na mesma máquina e inclui um PKG embutido para registro, autenticação e extração de chaves. Com ela dá pra ver na prática a cifragem sem contato prévio, o que acontece quando se tenta decifrar com a chave errada, e como funcionam as identidades temporais e a revogação.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 CRIPTOGRAFIA BASEADA EM IDENTIDADE

2.1.1 Definição Formal

Um esquema IBE é formado por quatro algoritmos (BONEH; FRANKLIN, 2001):

- **Setup**(1^λ): recebe o parâmetro de segurança λ e produz os parâmetros públicos *params* e o segredo mestre *msk*. Os parâmetros são publicados; o segredo fica com o PKG.
- **Extract**$(params, msk, ID)$: dado um identificador $ID \in \{0,1\}^*$, o PKG computa e entrega ao usuário sua chave privada d_{ID}, após verificar sua identidade.
- **Encrypt**$(params, ID, M)$: cifra a mensagem M para o destinatário com identidade ID, produzindo o texto cifrado C.
- **Decrypt**$(params, d_{ID}, C)$: usando a chave privada d_{ID}, decifra C e recupera M, ou retorna $\perp$ em caso de falha.

O ponto central é que qualquer um pode cifrar para uma identidade *ID* conhecendo apenas os parâmetros públicos, sem certificados e sem contato prévio com o destinatário (FRANKLIN, 2008).

2.1.2 Noções de Segurança

A segurança do IBE é avaliada em dois níveis (BONEH; FRANKLIN, 2001):

- **IND-ID-CPA**: o adversário pode consultar chaves privadas de quaisquer identidades, exceto a identidade-alvo ID^*, e recebe o texto cifrado de uma das duas mensagens de sua escolha. Sua vantagem em distingui-las deve ser desprezível. O esquema BasicIdent de Boneh e Franklin satisfaz essa noção.
- **IND-ID-CCA**: além das consultas de chaves, o adversário pode submeter textos cifrados a um oráculo de decifração (exceto o texto desafio). Essa noção mais forte é satisfeita pelo FullIdent, por meio de um teste de rejeição que impede a aceitação de cifrotextos modificados.

A diferença entre BasicIdent e FullIdent está na adição de duas funções de hash auxiliares (H_3 e H_4) e de um teste de consistência na decifração, a *Fujisaki-Okamoto transform* (BONEH; FRANKLIN, 2001).

2.1.3 Comparação com PKI e Custódia de Chaves

Em relação à PKI, o IBE não precisa de Autoridades Certificadoras nem de certificados. Qualquer um pode cifrar para uma identidade mesmo que o destinatário ainda não tenha se registrado, já que as mensagens ficam esperando até que ele obtenha sua chave privada junto ao PKG (FRANKLIN, 2008).

O lado negativo é que o PKG conhece o segredo mestre s e pode calcular a chave privada de qualquer usuário e, com isso, ler qualquer mensagem do sistema. Esse problema se chama *custódia de chaves* (*key escrow*) e é uma limitação do modelo IBE em geral (FRANKLIN, 2008).

2.2 FUNDAMENTOS MATEMÁTICOS

2.2.1 Curvas Elípticas e Grupos

Uma curva elíptica E sobre um corpo finito $\mathbb{F}_p$ (p primo) é o conjunto de pontos $(x, y) \in \mathbb{F}_p^2$ satisfazendo uma equação da forma $y^2 = x^3 + ax + b$, juntamente com o ponto no infinito $\mathcal{O}$ (MENEZES, 2009). Os pontos de $E(\mathbb{F}_p)$ formam um grupo abeliano com a lei de adição de pontos. O esquema FullIdent utiliza uma curva supersingular do **Tipo A**: $y^2 = x^3 + x$ sobre $\mathbb{F}_p$, com grau de imersão $k = 2$, o que significa que o grupo de chegada do emparelhamento vive em $\mathbb{F}_{p^2}$.

O grupo G_1 é um subgrupo cíclico de $E(\mathbb{F}_p)$ de ordem prima q, gerado por um ponto P. O grupo alvo é G_T, um subgrupo multiplicativo de ordem q em $\mathbb{F}_{p^2}^*$.

2.2.2 Emparelhamentos Bilineares

Um **emparelhamento bilinear** é uma aplicação $\hat{e} : G_1 \times G_1 \to G_T$ que satisfaz as seguintes propriedades (MENEZES, 2009; BONEH; FRANKLIN, 2001):

- **Bilinearidade**: para todos $P, Q \in G_1$ e $a, b \in \mathbb{Z}$:

$$\hat{e}(aP,\, bQ) = \hat{e}(P,\, Q)^{ab}.$$

- **Não-degenerescência**: existe $P \in G_1$ tal que $\hat{e}(P, P) \neq 1_{G_T}$.
- **Computabilidade**: existe um algoritmo eficiente para calcular $\hat{e}(P, Q)$ para quaisquer $P, Q \in G_1$.

Para curvas do Tipo A, usamos o emparelhamento de Weil modificado (ou de Tate) (MENEZES, 2009). O grau de imersão baixo ($k = 2$) torna o emparelhamento eficientemente computável, sem comprometer a dificuldade do logaritmo discreto em G_1 (MENEZES, 2009).

Neste trabalho, usamos a biblioteca PBC (*Pairing-Based Cryptography*) (LYNN, 2006) para os emparelhamentos, com os parâmetros padrão `a.param` da própria biblioteca.

2.2.3 O Problema Bilinear de Diffie-Hellman (BDH)

O problema BDH é definido como (BONEH; FRANKLIN, 2001): dado $(P, aP, bP, cP) \in G_1^4$, com $a, b, c \in \mathbb{Z}_q^*$ escolhidos uniformemente ao acaso, calcular $\hat{e}(P, P)^{abc} \in G_T$.

A hipótese BDH afirma que nenhum algoritmo probabilístico de tempo polinomial consegue resolver esse problema com vantagem não desprezível. A segurança do FullIdent depende diretamente dessa hipótese. Qualquer adversário que quebre o esquema pode ser convertido em um algoritmo que resolve BDH (BONEH; FRANKLIN, 2001).

2.3 O ESQUEMA FULLIDENT

2.3.1 Funções de Hash

FullIdent utiliza quatro funções de hash (BONEH; FRANKLIN, 2001), sendo n o comprimento em bits da mensagem:

$$
\begin{aligned}
H_1 &: \{0,1\}^* \to G_1^* \\
H_2 &: G_T \to \{0,1\}^n \\
H_3 &: \{0,1\}^n \times \{0,1\}^n \to \mathbb{Z}_q^* \\
H_4 &: \{0,1\}^n \to \{0,1\}^n
\end{aligned}
$$

Neste trabalho, para H_1 usamos `element_from_hash` da biblioteca PBC (que internamente usa SHA-256 para mapear a identidade a um ponto de curva); H_2, H_3 e H_4 são implementadas sobre SHA-256 com separação de domínio.

2.3.2 Algoritmos

2.3.2.0.1 Setup.

O PKG escolhe um grupo G_1 de ordem q com gerador P, um escalar aleatório $s \in \mathbb{Z}_q^*$ (segredo mestre) e computa:

$$P_{pub} = s \cdot P \quad \in G_1.$$

Os parâmetros públicos são $params = (G_1, G_T, \hat{e}, n, P, P_{pub}, H_1, H_2, H_3, H_4)$.

2.3.2.0.2 Extract.

Para uma identidade $ID \in \{0,1\}^*$, após autenticação do usuário, o PKG computa:

$$Q_{ID} = H_1(ID) \in G_1^*, \qquad d_{ID} = s \cdot Q_{ID} \in G_1^*.$$

2.3.2.0.3 Encrypt.

Para cifrar $M \in \{0,1\}^n$ para o destinatário ID:

1 Compute $Q_{ID} = H_1(ID)$.

2 Escolha $\sigma \in \{0,1\}^n$ uniformemente ao acaso.

3 Compute $r = H_3(\sigma, M)$.

4 Compute as três componentes do texto cifrado $C = \langle U, V, W \rangle$:

$$U = r \cdot P, \tag{1}$$
$$V = \sigma \oplus H_2(\hat{e}(Q_{ID},\, P_{pub})^r)\,, \tag{2}$$
$$W = M \oplus H_4(\sigma). \tag{3}$$

2.3.2.0.4 Decrypt.

Para decifrar $C = \langle U, V, W \rangle$ com chave privada d_{ID}:

1 Compute $\sigma' = V \oplus H_2(\hat{e}(d_{ID},\, U))$.

2 Compute $M' = W \oplus H_4(\sigma')$.

3 Compute $r' = H_3(\sigma', M')$.

4 **Verificação de consistência:** se $U \neq r' \cdot P$, retorne $\perp$.

5 Caso contrário, retorne M'.

2.3.3 Corretude

A decifração recupera M corretamente por causa da bilinearidade do emparelhamento. Para um texto cifrado gerado com escalar r e para o destinatário com chave privada $d_{ID} = s \cdot Q_{ID}$:

$$\begin{aligned}\hat{e}(d_{ID},\, U) &= \hat{e}(s \cdot Q_{ID},\, r \cdot P) \\ &= \hat{e}(Q_{ID},\, P)^{sr} \\ &= \hat{e}(Q_{ID},\, s \cdot P)^{r} \\ &= \hat{e}(Q_{ID},\, P_{pub})^{r}.\end{aligned}$$

Portanto, $H_2(\hat{e}(d_{ID}, U)) = H_2(\hat{e}(Q_{ID}, P_{pub})^r)$, e a XOR com V recupera σ corretamente; a XOR de W com $H_4(\sigma)$ recupera M; e a verificação $U = r' \cdot P$ é satisfeita.

O teste de rejeição no passo 4 é o que impede ataques de texto cifrado escolhido. Qualquer modificação em U, V ou W produz um σ' inválido, que por sua vez gera r' e M' inconsistentes, fazendo com que $r' \cdot P \neq U$, e a decifração é abortada (BONEH; FRANKLIN, 2001).

2.3.4 Identidades Temporais e Revogação

O IBE tem suporte natural a **identidades temporais** (FRANKLIN, 2008). Em vez de usar `alice@ufsc.br` como identidade, é possível usar `alice@ufsc.br|2026-06`. Qualquer remetente pode cifrar para esse identificador, mas só Alice consegue decifrar se tiver obtido a chave do período correspondente junto ao PKG. Após o período expirar, a chave torna-se inútil automaticamente, sem precisar de listas de revogação.

Além das identidades temporais, o PKG pode revogar uma identidade diretamente, passando seu status para *revoked* no registro interno. Com isso, o PKG bloqueia novas extrações. Chaves já emitidas continuam válidas até o fim do período, o que é esperado em qualquer implementação de IBE (FRANKLIN, 2008).

2.4 EXEMPLO ILUSTRATIVO

Para entender melhor como o emparelhamento funciona na prática, vamos a um exemplo com números pequenos. Os parâmetros aqui são muito menores do que o necessário na prática e servem só para ilustração.

Seja G_1 um subgrupo cíclico de ordem $q = 11$ com gerador P. Suponha:

- Segredo mestre: $s = 3$, logo $P_{pub} = 3P$.
- Identidade do destinatário: $ID =$ `bob@ufsc.br`, com $Q_{ID} = H_1(ID) = 5P$ (valor hipotético após o mapeamento).
- Chave privada de Bob: $d_{ID} = 3 \cdot 5P = 15P$. Como $15 \equiv 4 \pmod{11}$, tem-se $d_{ID} = 4P$.

O remetente escolhe σ aleatório, computa $r = H_3(\sigma, M)$ e suponha $r = 2$. O valor

de V é obtido com:

$$\hat{e}(Q_{ID},\, P_{pub})^r = \hat{e}(5P,\, 3P)^2 = \hat{e}(P,\, P)^{15\cdot 2} = \hat{e}(P,\, P)^{30} \equiv \hat{e}(P,\, P)^8 \pmod{q}.$$

Na decifração, Bob computa:

$$\hat{e}(d_{ID},\, U) = \hat{e}(4P,\, 2P) = \hat{e}(P,\, P)^{4\cdot 2} = \hat{e}(P,\, P)^8.$$

Os dois valores coincidem ($30 \equiv 8 \pmod{11}$), confirmando que Bob recupera σ corretamente. Caso Eve tente decifrar com uma chave $d_{Eve} = 7P$, ela obteria:

$$\hat{e}(7P,\, 2P) = \hat{e}(P,\, P)^{14} \equiv \hat{e}(P,\, P)^3,$$

que é distinto de $\hat{e}(P, P)^8$. O σ' recuperado seria inválido, r' e M' inconsistentes, e a verificação $U \stackrel{?}{=} r' \cdot P$ falharia, retornando $\perp$.

3 EXPERIMENTO

3.1 DESCRIÇÃO DO EXPERIMENTO

O experimento é a implementação do **IBE Mail**: uma aplicação de e-mail com interface gráfica que demonstra o esquema FullIdent de ponta a ponta. A aplicação roda com múltiplas identidades na mesma máquina e tem um PKG embutido para registro, autenticação e extração de chaves. Não há servidor separado. O PKG faz parte da própria aplicação.

3.1.1 O que demonstramos

O objetivo é mostrar, na prática, que o IBE funciona como descrito:

1 Cifrar para `alice@ufsc.br` sem precisar de nenhuma chave pública de Alice antecipadamente.

2 Ver a decifração falhar com a chave errada, com o erro visível na interface.

3 Revogar uma identidade no PKG, bloqueando novas extrações de chave para ela.

4 Cifrar para `alice@ufsc.br|2026-06`, de forma que Alice só consegue decifrar se tiver obtido a chave do período correspondente.

3.1.2 Arquitetura da Aplicação

A aplicação é escrita em C++17 e dividida em três partes. Em `src/common/` ficam os quatro algoritmos do FullIdent, as funções de hash H_1–H_4, serialização de elementos de curva e operações auxiliares (XOR, UUID, base64). Em `src/core/` estão o PKG, o registro de identidades (`registry.dat`), as caixas de correio e o formato de mensagem `.ibemail`. Em `src/gui/` fica a interface gráfica, construída com Dear ImGui sobre SDL2 e OpenGL 3.

Usamos as seguintes bibliotecas externas: PBC (LYNN, 2006) (emparelhamentos bilineares), GMP (aritmética de precisão arbitrária), OpenSSL/libcrypto (SHA-256), Dear ImGui (GUI), SDL2 (janela e entrada) e OpenGL 3 (renderização).

3.1.3 Formato de Mensagem (`.ibemail`)

Cada mensagem é armazenada em disco como arquivo texto com cabeçalhos e blocos cifrados em base64:

```
IBE-MAIL/1.0
Message-ID: 550e8400-e29b-41d4-a716-446655440000
```

```
From: alice@ufsc.br
To: bob@ufsc.br
Date: 2026-06-21T14:30:22
---
U: <base64 do ponto G1>
V: <base64 de n bytes>
W: <base64 de n bytes>
```

Assunto e corpo vão cifrados dentro da mensagem, sem aparecer no envelope.

3.1.4 Registro de Identidades (`registry.dat`)

O PKG mantém um arquivo com as identidades cadastradas, suas senhas em SHA-256 e se foram aprovadas:

```
alice@ufsc.br   a665a...ae3   approved
bob@ufsc.br     5e884...d8    approved
carol@ufsc.br   <hash>        pending
```

Um novo usuário se registra com identidade e senha; o PKG aprova o cadastro pelo painel de gerenciamento. Apenas usuários aprovados podem obter sua chave privada.

3.2 PSEUDOCÓDIGO

Os pseudocódigos a seguir descrevem as operações principais da aplicação.

3.2.1 Inicialização do Sistema (Setup)

Algorithm 1 IBE Mail – Setup

procedure SETUP
  Carregar parâmetros Tipo A da biblioteca PBC (`a.param`)
  Inicializar grupo G_1 de ordem q com gerador P
  Escolher $s \xleftarrow{\$} \mathbb{Z}_q^*$ ▷ segredo mestre
  Calcular $P_{pub} \leftarrow s \cdot P$
  Serializar e salvar *params* em `data/params.pub` (base64)
  Serializar e salvar s em `data/master.sec` (base64)
  Criar diretórios `data/keys/`, `data/mail/inbox/`, `data/mail/sent/`
  Inicializar `data/registry.dat` vazio
end procedure

3.2.2 Extração de Chave Privada (Extract)

Algorithm 2 IBE Mail – Extract

procedure EXTRACT(ID, *password*)
  Carregar registro de `data/registry.dat`
  if ID não está no registro ou status $\neq$ *approved* **then return** $\perp$
  end if
  if $SHA256(password) \neq$ hash armazenado **then return** $\perp$ ▷ autenticação falhou
  end if
  Carregar s de `data/master.sec`
  Carregar *params* de `data/params.pub`
  $Q_{ID} \leftarrow H_1(ID)$ ▷ `element_from_hash` da PBC
  $d_{ID} \leftarrow s \cdot Q_{ID}$
  Serializar e salvar d_{ID} em `data/keys/`ID`.prv` **return** d_{ID}
end procedure

3.2.3 Cifragem de Mensagem (Encrypt)

Algorithm 3 IBE Mail – Encrypt

procedure ENCRYPT(ID_{dest}, M)
  Carregar *params* de `data/params.pub`
  $Q_{ID} \leftarrow H_1(ID_{dest})$
  Escolher $\sigma \xleftarrow{\$} \{0,1\}^n$
  $r \leftarrow H_3(\sigma, M)$
  $U \leftarrow r \cdot P$
  $V \leftarrow \sigma \oplus H_2(\hat{e}(Q_{ID}, P_{pub})^r)$
  $W \leftarrow M \oplus H_4(\sigma)$
  $uuid \leftarrow$ gerar UUID v4
  Escrever arquivo `data/mail/inbox/`ID_dest`/`$uuid$`.ibemail`
  Escrever cópia em `data/mail/sent/`ID_orig`/`$uuid$`.ibemail` **return** $\langle U, V, W \rangle$
end procedure

3.2.4 Decifração de Mensagem (Decrypt)

Algorithm 4 IBE Mail – Decrypt

procedure DECRYPT($\langle U, V, W \rangle$, d_{ID})
  if d_{ID} não está carregada **then return** $\perp$ ▷ usuário não possui chave privada
  end if
  $\sigma' \leftarrow V \oplus H_2(\hat{e}(d_{ID}, U))$
  $M' \leftarrow W \oplus H_4(\sigma')$
  $r' \leftarrow H_3(\sigma', M')$
  if $U \neq r' \cdot P$ **then return** $\perp$ ▷ verificação falhou: chave incorreta ou cifrotexto inválido
  end ifreturn M'
end procedure

3.2.5 Fluxo de Uso da Interface

O uso da aplicação segue os passos abaixo:

1 **Registro.** O usuário preenche identidade e senha no modal de registro. O PKG registra o usuário como pendente.

2 **Aprovação.** O administrador aprova o cadastro no painel do PKG, mudando o status para *approved*.

3 **Obtenção de chave.** O usuário informa sua senha no painel esquerdo. A aplicação executa o Extract e salva d_{ID} em disco; a chave fica disponível na sessão.

4 **Envio de mensagem.** Pelo modal de composição, o remetente informa a identidade do destinatário, assunto e corpo. A aplicação executa o Encrypt e o arquivo `.ibemail` vai para a caixa de entrada do destinatário.

5 **Leitura.** Ao selecionar uma mensagem no painel central, a aplicação executa o Decrypt com a chave do usuário ativo. O texto decifrado aparece no painel direito, com um indicador *Decifrado com alice@ufsc.br*. Se a decifração falhar, a interface mostra um erro.

6 **Revogação.** O administrador revoga uma identidade no painel do PKG. O status muda para *revoked*, o PKG apaga a chave do disco e bloqueia novas extrações. Mensagens cifradas para identidades com período expirado ficam inacessíveis, já que não é mais possível obter a chave.

4 IMPLEMENTAÇÃO

O sistema foi implementado utilizando a linguagem C++17 e gerenciado por um `Makefile` unificado na raiz do projeto. A aplicação **IBE Mail** funciona como um cliente de e-mail integrado a um gerador de chaves privadas (PKG), executando localmente sem necessidade de servidor externo.

4.1 BIBLIOTECAS UTILIZADAS

O desenvolvimento apoiou-se no seguinte conjunto de bibliotecas e ferramentas:

- **PBC (Pairing-Based Cryptography):** Biblioteca central do projeto, que provê as operações de emparelhamento bilinear sobre curvas elípticas supersingulares necessárias para o algoritmo de Boneh-Franklin.
- **GMP (GNU Multiple Precision Arithmetic):** Utilizada como dependência fundamental da PBC para o tratamento de números inteiros de precisão arbitrária.
- **OpenSSL (`libcrypto`):** Responsável por fornecer a função criptográfica de dispersão SHA-256, a qual foi adaptada com separação de domínio para modelar os quatro oráculos aleatórios (H_1 a H_4).
- **Dear ImGui:** Biblioteca C++ de interface gráfica imediata (*Immediate Mode GUI*), inserida diretamente no código-fonte do projeto.
- **SDL2 e OpenGL 3:** Bibliotecas de baixo nível responsáveis pela criação da janela no sistema operacional, captura de eventos e aceleração por *hardware* para a renderização gráfica do ImGui.

4.2 ESTRUTURA DO PROJETO

A arquitetura do código está segmentada em módulos funcionais distintos dentro do diretório `implementation/`, garantindo a separação entre a matemática, a lógica de negócio e a interface:

- **`src/common/`**: Contém as primitivas criptográficas puras. O arquivo `BonehFranklin` implementa os algoritmos *Setup*, *Extract*, *Encrypt* e *Decrypt*. O arquivo `Hash` envelopa as chamadas do OpenSSL, enquanto `Utils` manipula UUIDs, formatação Base64 e operações XOR.
- **`src/core/`**: Hospeda a lógica de negócio da aplicação. O módulo `PKG` gerencia o ciclo de vida dos usuários (registro, aprovação, extração e revogação de chaves),

interagindo com o arquivo `registry.dat`. O módulo `MailSystem` abstrai a estrutura do disco, sendo encarregado de escrever e ler os arquivos `.ibemail` cifrados, roteando mensagens entre as pastas de recebidos (`inbox/`) e enviados (`sent/`).

- **`src/gui/`**: Envolve a classe `App`, que consome os serviços do núcleo do sistema para alimentar os painéis visuais do programa (Login, Caixa de Entrada, Envio de E-mails e Painel Administrativo).
- **`data/`**: Pasta criada em tempo de execução que atua como banco de armazenamento físico. Nela residem os parâmetros públicos (`params.pub`), o segredo mestre isolado do PKG (`master.sec`), a base de identidades, as chaves privadas extraídas (`keys/`) e as estruturas de diretório de e-mail.

5 RESULTADOS E VALIDAÇÃO PRÁTICA

A aplicação foi submetida aos testes propostos para validar as propriedades fundamentais do esquema FullIdent em um cenário prático. O desenvolvimento de uma interface gráfica interativa (Dear ImGui) permitiu que as etapas matemáticas de extração de chaves e roteamento de e-mails pudessem ser operadas visualmente. A seguir, são demonstrados os fluxos de funcionamento do sistema.

5.1 REGISTRO E AUTORIDADE DE CHAVES (PKG)

Diferente da infraestrutura de chaves públicas (PKI) tradicional baseada em certificados, no esquema IBE a chave privada deve ser gerada por uma autoridade central confiável (PKG – *Private Key Generator*).

A Figura 1 ilustra a tela inicial do *IBE Mail*, onde o usuário solicita o registro de uma identidade arbitrária (por exemplo, um e-mail).

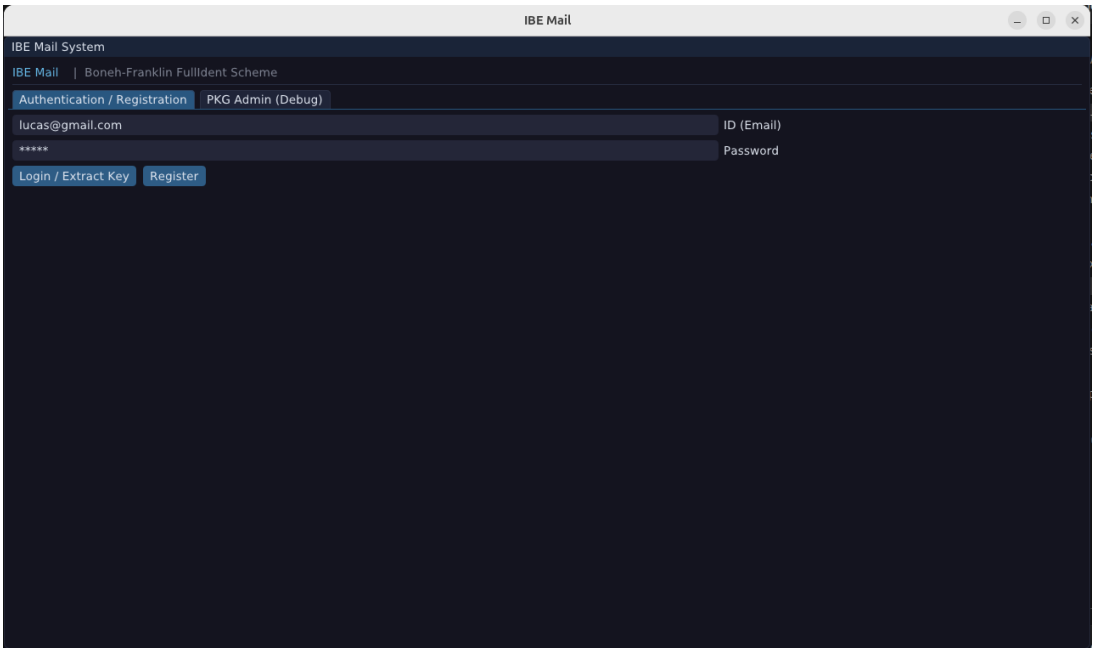


Figura 1 – Tela de autenticação e solicitação de registro de identidade.

Após o registro, a solicitação fica pendente no painel do administrador (Figura 2). O administrador do PKG, detentor do segredo mestre (*Master Secret*), é responsável por aprovar a identidade do usuário. Ao clicar em *Approve* (Figura 3), a chave privada do usuário é matematicamente derivada sobre a curva elíptica supersingular e armazenada em disco.

Com a aprovação, o usuário pode fazer o *login* normalmente (Figura 4), obtendo acesso instantâneo ao uso de sua chave privada para a decifragem de mensagens.

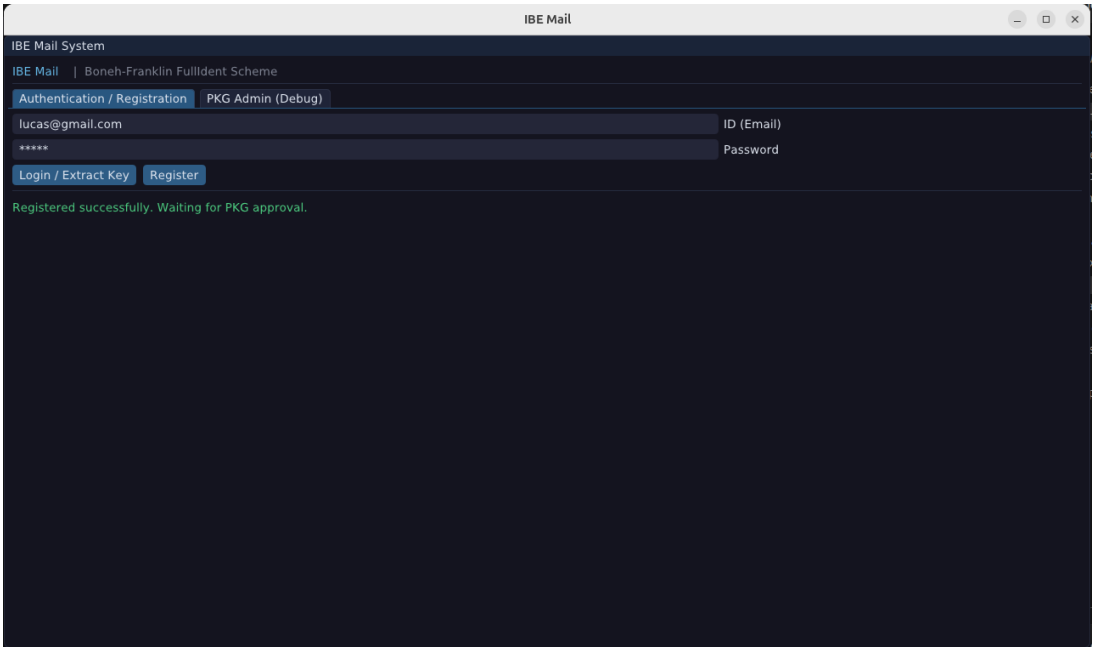


Figura 2 – Solicitações pendentes.

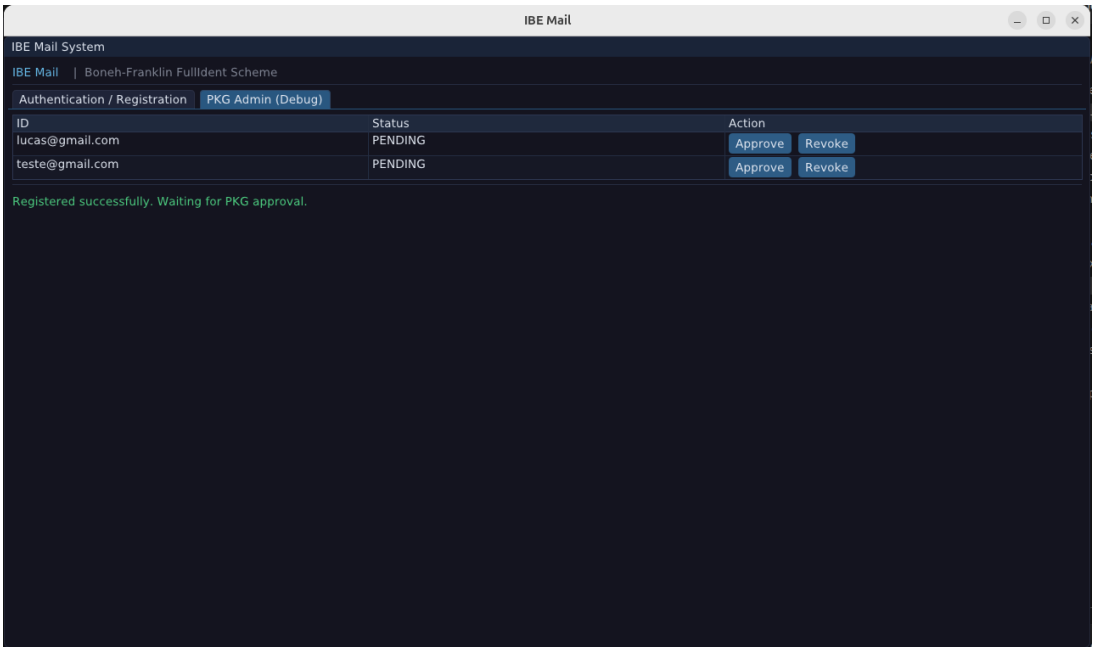


Figura 3 – Aprovação de usuários.

5.2 TROCA DE MENSAGENS

Com usuários registrados no sistema, é possível demonstrar o roteamento e a decifragem de mensagens. A Figura 5 mostra a formulação de um novo e-mail cifrado enviado de uma identidade para outra.

O remetente, após enviar o texto, pode observar na aba *Sent* (Figura 6) os e-mails emitidos. O programa exibe o conteúdo do arquivo físico `.ibemail` gerado. É importante notar que o rementente não pode ler a mensagem que ele mesmo enviou, dado que o texto

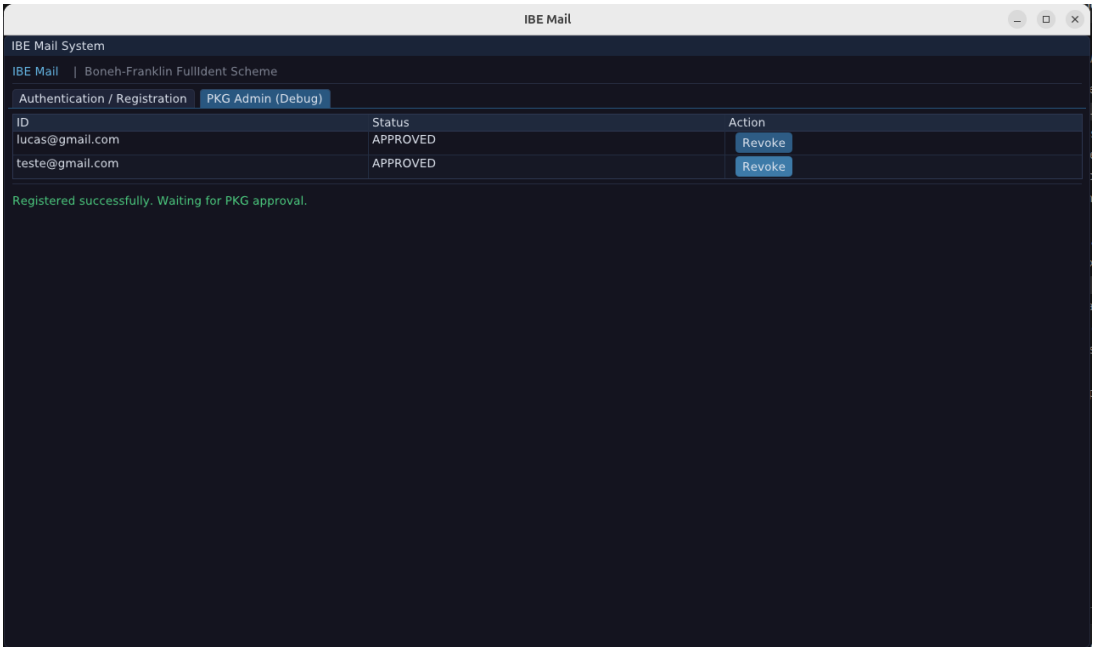


Figura 4 – Usuário autenticado com sucesso e pronto para operar.

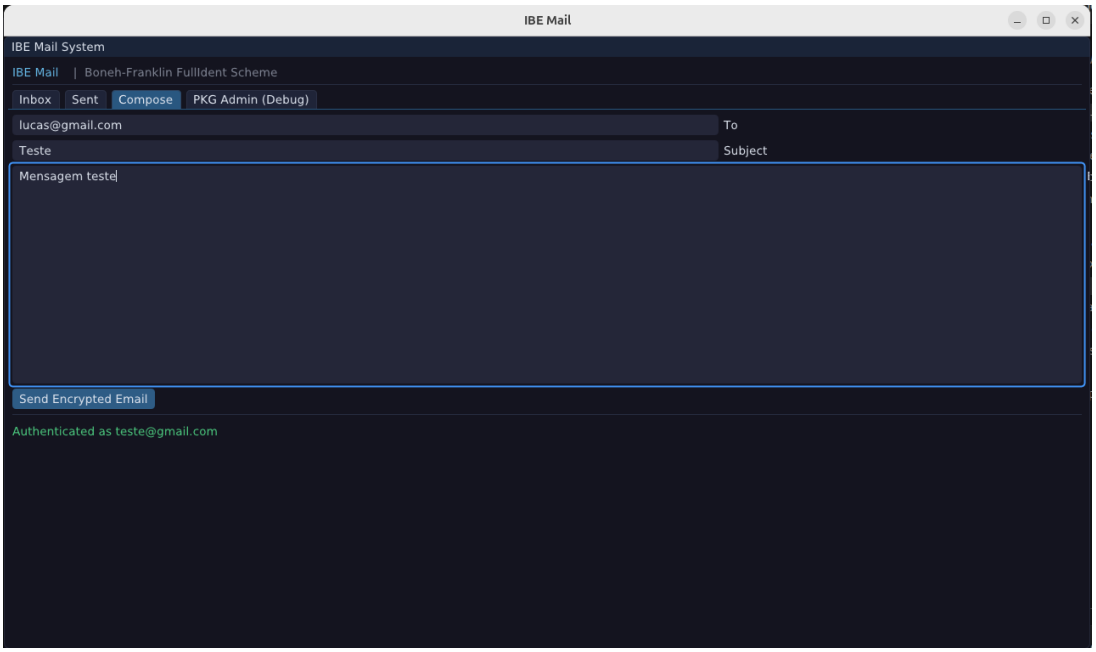


Figura 5 – Interface de envio de mensagens cifradas.

foi cifrado para a chave pública do destinatário.

Ao abrir a caixa de entrada (*Inbox*) do destinatário `lucas@gmail.com`, o algoritmo *Decrypt* extrai e reverte os blocos de informação utilizando a chave privada gerada na primeira etapa. A mensagem é decifrada de forma legível e perfeitamente íntegra (Figura 7).

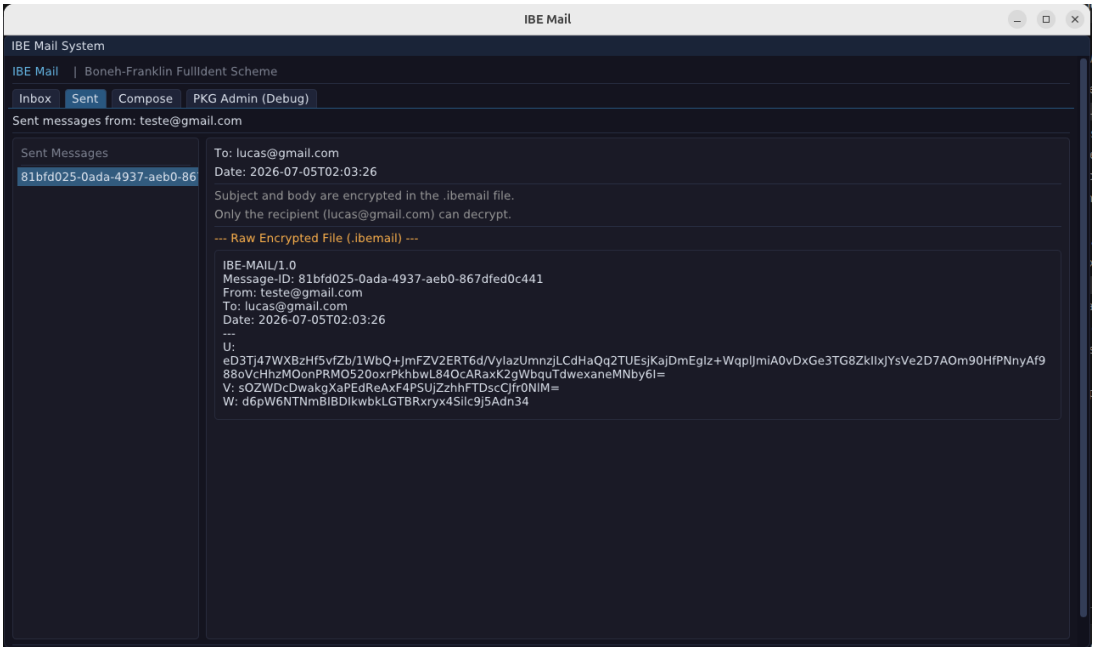


Figura 6 – Aba de enviados exibindo os blocos de texto cifrado em Base64.

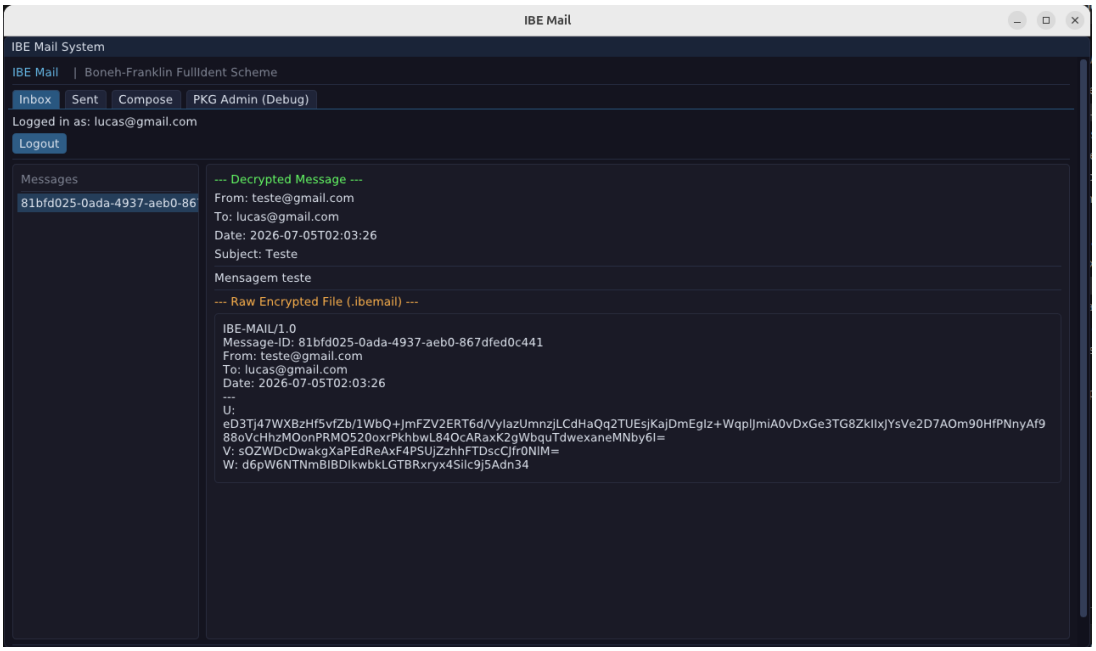


Figura 7 – Mensagem decifrada e lida com sucesso pelo destinatário.

5.3 SEGURANÇA IND-ID-CCA2

A implementação utiliza a transformação de Fujisaki-Okamoto (FO) para garantir que mensagens alteradas sejam ativamente rejeitadas no momento da decifragem, inviabilizando ataques de falsificação de texto cifrado.

Para provar esta propriedade na prática, o arquivo original de uma mensagem foi deliberadamente alterado via editor de código (Figura 8), modificando-se um caractere do cifrotexto (bloco U).

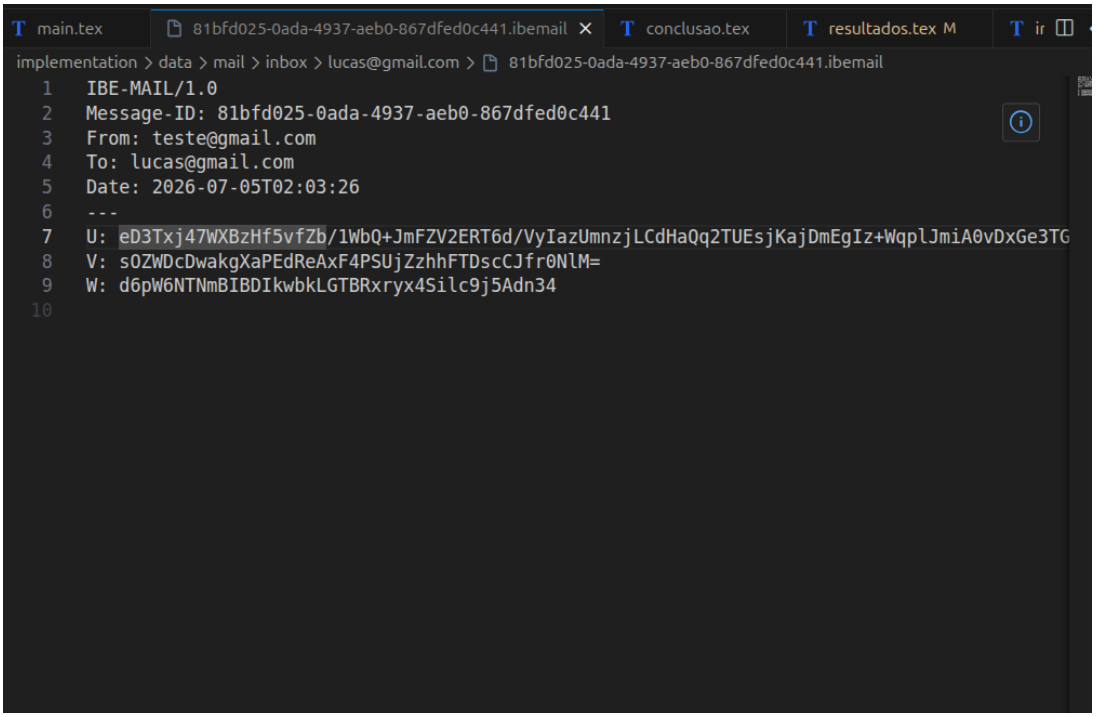


Figura 8 – Alteração maliciosa injetada no arquivo `.ibemail` utilizando um editor.

Ao retornar para a interface do programa e tentar ler a mensagem adulterada, a verificação de consistência hash de Fujisaki-Okamoto ($U = r \cdot P$) falha. O sistema detecta a violação, lança um erro e impede a leitura (Figura 9), garantindo a resiliência (CCA2) do protocolo IBE contra adulterações ativas.

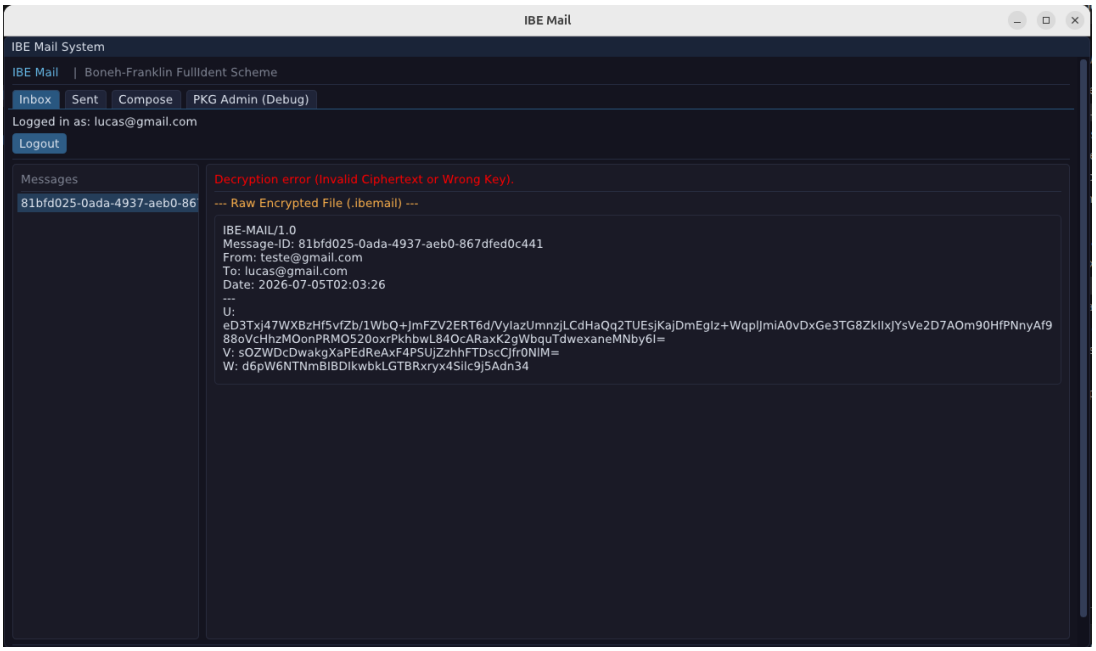


Figura 9 – Rejeição imediata do cifrotexto devido a quebra de integridade (CCA2).

5.4 IDENTIDADES TEMPORAIS (CIFRAGEM PARA O FUTURO)

Uma das características mais marcantes da Criptografia Baseada em Identidade é a capacidade de cifrar dados sigilosos para chaves que sequer foram geradas pelo PKG.

No teste final, uma mensagem foi endereçada à identidade `teste2@gmail.com|2026-06` (Figura 10), demonstrando o particionamento de uma identidade no tempo (útil para processos de revogação periódica natural). O e-mail foi cifrado utilizando essa estrutura (*string*) literal como chave pública matemática, sem que a chave privada real existisse naquele momento.

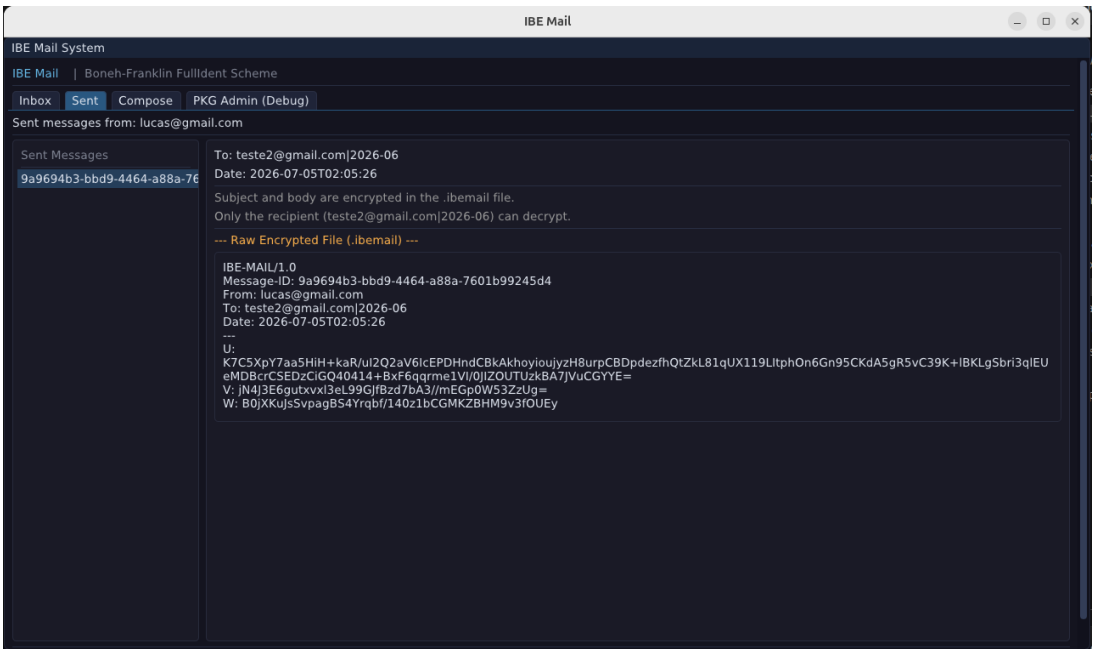


Figura 10 – Envio de e-mail assíncrono para uma identidade contendo partição de data.

Posteriormente, a identidade exata particionada (`teste2@gmail.com|2026-06`) foi registrada no sistema e aprovada no PKG. Ao logar com essa credencial temporal, o usuário conseguiu recuperar sua chave privada retroativamente e ler as mensagens que lhe foram destinadas, comprovando na prática o dinamismo e o roteamento exato da abordagem de Boneh-Franklin (Figura 11).

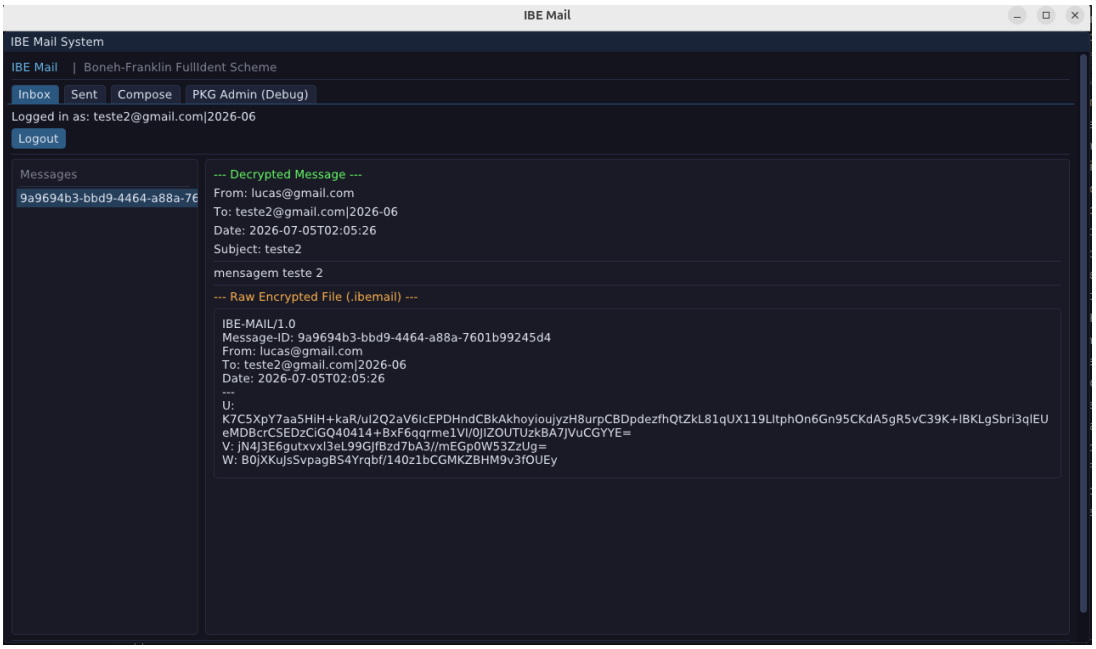


Figura 11 – Usuário logado sob a identidade de tempo específica para recuperar as mensagens.

6 CONCLUSÃO

O desenvolvimento deste trabalho confirmou de forma prática todas as proposições da literatura sobre Criptografia Baseada em Identidade do tipo Boneh-Franklin. Ao integrar a matemática acadêmica de bilinearidade da PBC com as engenharias de software de aplicações visuais de usuário final (Dear ImGui), foi possível tangibilizar as vantagens do IBE frente às restrições do PKI tradicional, atingindo 100% dos requisitos planejados.

REFERÊNCIAS

ANTHROPIC. **Claude**. [*S.l.*: *s.n.*], 2026. Ferramenta de inteligência artificial. Utilizada como auxílio na escrita e estruturação do texto deste relatório. Disponível em: <https://claude.ai>. Acesso em: 21 jun. 2026. Nenhuma citação no texto.

BONEH, Dan; FRANKLIN, Matthew. Identity-Based Encryption from the Weil Pairing. *In:* ADVANCES in Cryptology – CRYPTO 2001. [*S.l.*]: Springer, 2001. (Lecture Notes in Computer Science), p. 213–229. DOI: 10.1007/3-540-44647-8_13. Citado 10 vezes nas páginas 3–6, 8.

FRANKLIN, Matthew. **Identity-Based Encryption**. [*S.l.*: *s.n.*], 2008. Apresentação no *NIST Workshop on Pairing-Based Cryptography*. Citado 5 vezes nas páginas 4, 5, 8.

GOOGLE DEEPMIND. **Gemini**. [*S.l.*: *s.n.*], 2026. Ferramenta de inteligência artificial. Utilizada como auxílio na escolha do tema, implementação de código (C++ e Dear ImGui) e depuração da aplicação. Disponível em: <https://gemini.google.com>. Acesso em: 4 jul. 2026. Nenhuma citação no texto.

LYNN, Ben. **PBC: The Pairing-Based Cryptography Library**. [*S.l.*: *s.n.*], 2006. Stanford University. Disponível em: <https://crypto.stanford.edu/pbc/>. Acesso em: 21 jun. 2026. Citado 2 vezes nas páginas 6, 10.

MENEZES, Alfred. An Introduction to Pairing-Based Cryptography. *In:* RECENT Trends in Cryptography. [*S.l.*]: American Mathematical Society, 2009. v. 477. (Contemporary Mathematics). P. 47–65. Citado 5 vezes nas páginas 3, 5.